Analysis of Length Normalization in End-to-End Speaker Verification System

Weicheng Cai[1], *Jinkun Chen,*[1], *Ming Li,*[1,2]

[1]School of Electronics and Information Technology, Sun Yat-sen University, Guangzhou, China
[2]Data Science Research Center, Duke Kunshan University, Kunshan, China

ml442@duke.edu

Abstract

Length normalization has been proved to be an effective strategy in i-vector based speaker verification systems. This simple non-linear transformation on i-vector allows the use of probabilistic models with Gaussian assumptions, and Gaussian-PLDA preceded by a length normalization has been the de factor standard back-end modeling in speaker verification system. In this paper, we present similar analysis of length normalization strategy but towards end-to-end scenario. We add a simple L_2-normalization layer followed by a scale layer after the penultimate layer of the end-to-end neural network. Therefore, the neural network can learn the length-normalized feature embedding in an end-to-end manner. We conducted experiment on the verification task of Voxceleb dataset. The results show that integrating this simple yet elegant step in the end-to-end training pipeline significantly boosts the performance of speaker verification. In testing stage of L_2-normalized end-to-end system, a simple inner product can achieve state-of-the-art result.

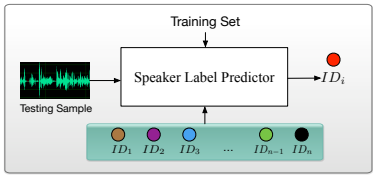
Index Terms: speaker verification, end-to-end, length normalization

1. Introduction

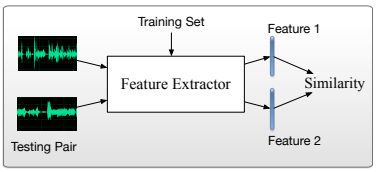
Text-independent speaker recognition (SR) tasks can be defined as an utterance level "sequence-to-one" learning issue, in compared with automatic speech recognition, which is a "sequence-to-sequence" tagging task. They are problems in that we are trying to retrieve information about an entire utterance rather than specific word content [1]. Moreover, there is no constraint on the lexicon words thus the training utterances and testing segments may have completely different contents [2]. The goal, therefore, might be to find a robust and time-invariant utterance level vector representation describing the distribution of the given input data sequence commonly with variable-length [3, 4].

Typically, SR can be categorized as speaker identification (SID) and speaker verification (SV) [5]. The former classifies a speaker to a specific identity, while the latter determines whether a pair of utterances belongs to the same person. In terms of the testing protocol, SR can be evaluated under closed-set or open-set settings, as illustrated in Fig. 1. For closed-set protocol, all testing identities are enrolled in the training set. It is natural to classify a testing utterane to a given identity. Therefore, closed-set SID can be well addressed as a classification problem. For the open-set protocol, speaker identities in testing set are usually disjoint from the ones in training set, which makes the SV more challenging yet closer to practice. Since it is impossible to classify testing utterances to known identities in training set, we need to map speakers to a discriminative feature space.

In this scenario, open-set SV is essentially a metric learning problem, where the key is to learn discriminative large-margin



(a) Closed-set speaker identification



(b) Open-set speaker verification

Figure 1: *Comparison of closed-set identification and open-set verification problem. The closed-set identification is equivalent to classification task, while the open-set verification can be considered as a metric learning task*

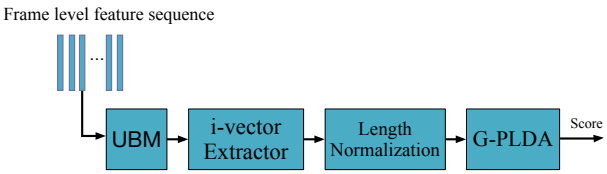


Figure 2: *i-vector PLDA based speaker verification system*

features. Recently years, the front-end i-vector representation [6] and back-end Probabilistic Linear Discriminant Analysis (PLDA) scoring [7, 8] have shown state-of-the-art performance. As demonstrated in Fig. 2, firstly, a variable-length feature sequence is extracted from raw audio signal. Then, selected feature frames in training dataset are grouped together to estimate a Gaussian Mixture Model (GMM) based universal background model (UBM). Sufficient statistics of each utterance on the UBM is accumulated, and a factor analysis based i-vector extractor is trained to project the statistics into low dimensional total variability space.

Once i-vector is extracted, additional feature comparison step is required to get the ultimate pairwise similarity score. In typical SV system, length normalization is a critical step for boosting the performance [9, 10]. This non-linear transformation allows the use of probabilistic models with Gaussian assumptions that yield equivalent performance to that of more complicated systems based on Heavy-Tailed assumptions [11]. Specially, Gaussian PLDA (G-PLDA) preceded by a length normalization has been the de factor standard in SV system [9].

The i-vector approaches are comprised of a series hand-

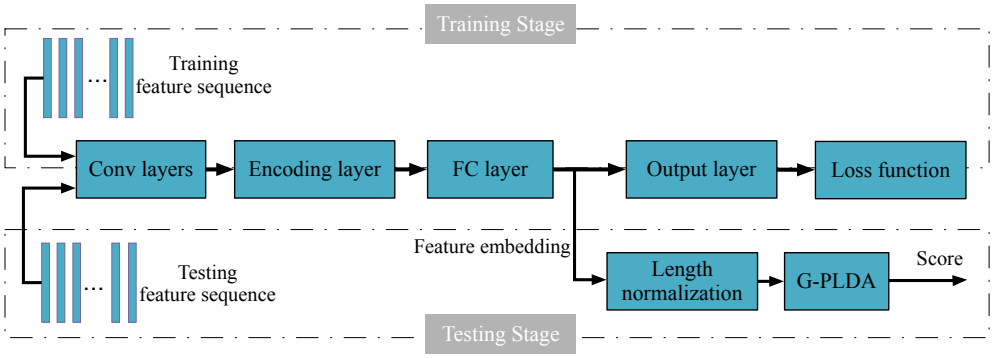


Figure 3: *Our end-to-end baseline system for speaker verification*

crafted or ad-hoc algorithmic components, and they show strong generalization ability and robustness when data and computational resource are limited. In recent years, with the merit of large labeled datasets, enormous computation capability, and effective network architectures, emerging progress towards end-to-end learning opens up a new area for exploration [3, 4, 12, 13, 14, 15, 16, 17, 18].

But in most of the aforementioned neural netowrk, inner product without normalization is adopted in training stage, and additional length normalization step in testing stage is necessary. This motivates us to introduce and study the effect of normalization during end-to-end training stage. Alghough the authors in [16, 17, 18] have introduced length normalization layer in their triplet loss based network architecture, there is no very clear analysis about its general usage on classification network. When we attempt to add L_2-normalization layer in our softmax loss trained neural network, we find it is non-trivial. In the subsequent section, we will identify and study the issues related to length normalization in end-to-end learning scheme through mathematic and concrete analysis, which yields understanding and helps with parameter settings.

2. Method

2.1. End-to-end baseline system

Our end-to-end baseline system is shown in Fig. 3. It accepts variable-length input data sequence and produces an utterance level result. All the components in the pipeline are jointly learned in an end-to-end manner with a unified loss function. Compared with i-vector approach, the end-to-end neural netowrk can be regarded as a supervised version of feature extractor to get the discriminative feature embedding.

Given input feature sequence such as log mel-filterbank energies (Fbank), the front-end convolution layers operate in an sliding window manner acting as a automatic local feature extractor. The frame level representation after the convolutional layers is still with temporal order. The remaining issue is to aggregate them together over the entire utterance. In this way, the encoding layer plays a role in extracting a fixed-dimentional utterance level representation from a variable-length input sequence. The utterance level representation is further transformed by a fully-connected (FC) layer. The basic softmax loss is adopted as our loss function:

$$\ell_s = -\frac{1}{M}\sum_{i=1}^{M}\log\frac{e^{\boldsymbol{W}_{y_i}^T f(\boldsymbol{x}_i)+\boldsymbol{b}_{y_i}}}{\sum_{j=1}^{C} e^{\boldsymbol{W}_j^T f(\boldsymbol{x}_j)+\boldsymbol{b}_j}} \tag{1}$$

where M is the training batch size, $\boldsymbol{x_i}$ is the ith input data sequence in the batch, $f(\boldsymbol{x_i})$ is the corresponding output of the penultimate layer of the end-to-end neural network, y_i is the corresponding target label, and $\boldsymbol{W}$ and $\boldsymbol{b}$ are the weights and bias for the last layer of the network which acts as a classifier.

Given feature embedding extracted from the output of FC layer, as in typical i-vector approach, additional length normalization and G-PLDA is adopted.

2.2. L_2-normalized end-to-end system

A naive practice is just add a L_2-normalization layer before output layer. However, we find the training process doesn't converge and lead to very poor performance result.

Similar issue has been studied recently by computer vision community [19, 20]. The effect of L_2-normalization layer is equivalent to add the L_2-constraint on the softmax loss. We need additional scale parameter to control the regularization level. Its mathematical formulation is given by

$$\begin{aligned} &\min -\frac{1}{M}\sum_{i=1}^{M}\log\frac{e^{\boldsymbol{W}_{y_i}^T f(\boldsymbol{x}_i)+\boldsymbol{b}_{y_i}}}{\sum_{j=1}^{C} e^{\boldsymbol{W}_j^T f(\boldsymbol{x}_i)+\boldsymbol{b}_j}} \\ &\text{subject to } \|f(\boldsymbol{x}_i)\|_2 = \alpha \end{aligned} \tag{2}$$

where $\boldsymbol{x}_i$ is the input sequence in a mini-batch of size M, y_i is the corresponding target label, $f(\boldsymbol{x}_i)$ is the feature descriptor obtained from the penultimate layer of end-to-end network, C is the number of subject classes, and $\boldsymbol{W}$ and $\boldsymbol{b}$ are the weights and bias for the last layer of the network which acts as a classifier. α is the scaling parameter to provide suitable bounds.

The whole network architecture is demonstrated in Fig. 4. The L_2- normalization layer normalizes the output of FC layer $\boldsymbol{x}$ to a unit vector.

$$\boldsymbol{e} = \frac{\boldsymbol{x}}{\|\boldsymbol{x}\|_2} \tag{3}$$

The scale layer scales the input unit vector to a fixed radius given by the parameter α.

$$\boldsymbol{s} = \alpha \times \boldsymbol{e} \tag{4}$$

In total, we just introduce one scalar parameter α which can be trained along with the other parameters of the network. The module is fully differentiable and can be used in the end-to-end training of the network. At test time, the feature embedding is extracted after the L_2-normalization layer. Since the features have already been normalized to unit length, a simple inner product is adopted to get the final similarity score.

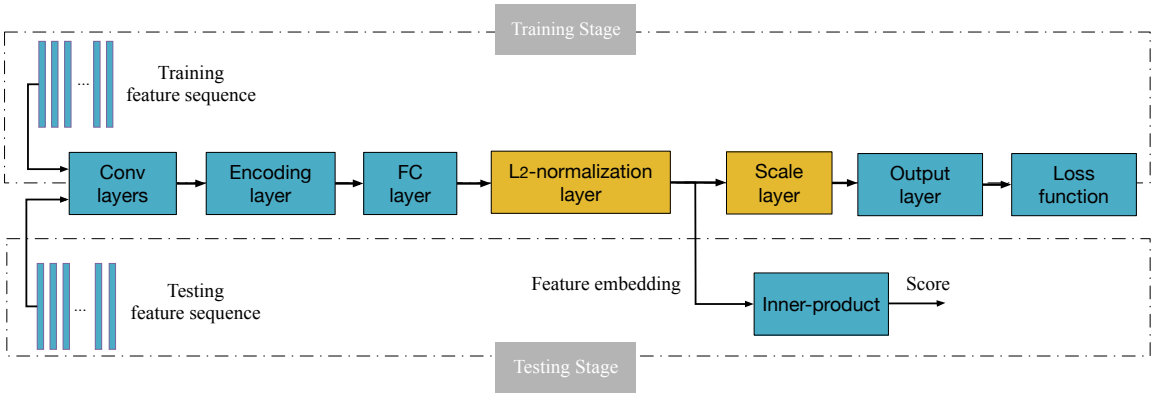


Figure 4: *L_2-normalized end-to-end speaker verification system*

The scaling parameter α plays a crucial role in deciding the performance. There are two ways to enforce the L_2-constraint: 1) by keeping α fixed throughout the training, and 2) by letting the network to learn the parameter α. But, the α parameter learned by the network is high which results in a relaxed L_2-constraint. The softmax classifier aimed at increasing the feature norm for minimizing the overall loss, increases the α parameter instead, allowing it more freedom to fit to the easy samples. Hence, α learned by the network forms an upper bound for the parameter. A better performance is obtained by fixing α to a lower constant value [19].

A simple L_2-normalization layer without scale layer is equivalent to the condition when $\alpha = 1$. The reason of poor results might be that a hypersphere with small radius has limited surface area for embedding features from the same class together and those from different classes far from each other.

Given the number of classes C for a dataset, to achieve a classification probability score of p, the authors in [19] give the formulation of theoretical lower bound on α by

$$\alpha_{low} = \log \frac{p(C-2)}{1-p} \tag{5}$$

3. Experiments

3.1. Data description

Voxceleb is a large scale text-independent SR dataset collected "in the wild", which contains over 100,000 utterances from 1251 celebrities [21]. We focus on its open-set verification task.

We pool the official split training and validation set together as our development dataset. There are totally 1211 celebrities in the development dataset. The testing dataset contains 4715 utterances from the rest 40 celebrities. There are totally 37720 pairs of trials including 18860 pairs of true trials. Two key performance metrics C_{det} [22] and Equal Error Rate (EER) are used to evaluate the system performance.

3.2. i-vector system

We build a referenced i-vector system based on Kaldi toolkit [23]. Firstly, 20 dimensional MFCC is augmented with their delta and double delta coefficients, making 60 dimensional MFCC feature vectors. Then, a frame-level energy-based voice activity detection (VAD) selects features corresponding to speech frames. A 2048 components full covariance GMM UBM is trained, along with a 600 dimensional i-vector extractor and full rank PLDA.

3.3. End-to-end system

Audio is converted to 64-dimensional Fbank with a frame-length of 25 ms, mean-normalized over a sliding window of up to 3 seconds. The same VAD processing as in i-vector system is used here.

In order to get higher abstract representation better for utterances with long duration, we design a deep CNN based on the well-known ResNet-34 layer architecture [24], as is described in Table 1. After the front-end CNN module, we adopt the simplest temporal average pooling (TAP) layer as our encoding layer. The input Fbanks have the shape of $64 \times L$, and finally 128 dimensional feature embeddings are extracted from the output of FC layer.

For L_2-normalized end-to-end system, we just add a simple L_2-normalization layer followed by a scale layer after the FC layer. All the other components is the same as the baseline end-to-end system.

To efficiently train the system, we set the mini-batch size of 192 in data parallelism over 4 GPUs used. The network is trained using typical stochastic gradient descent with momentum 0.9 and weight decay 1e-4. The learning rate is set to 0.1, 0.01, 0.001 and is switched when the training loss plateaus. The total number of epochs is 40. Since we have no separated validation set, the converged model after the last optimization setp is used for evaluation. For each training step, an integer L within $[300, 800]$ interval is randomly generated, and each data in the mini-batch is cropped or extended to L frames.

Table 1: *Our end-to-end baseline network configuration*

Layer	Output size	Downsample	Channels	Blocks
Conv1	$64 \times L_{in}$	False	16	-
Res1	$64 \times L_{in}$	False	16	3
Res2	$32 \times \frac{L_{in}}{2}$	True	32	4
Res3	$16 \times \frac{L_{in}}{4}$	True	64	6
Res4	$8 \times \frac{L_{in}}{8}$	True	128	3
Avgpool	$1 \times \frac{L_{in}}{8}$	-	128	-
Reshape	$128\times L_{out}, L_{out} = \frac{L_{in}}{8}$	-	-	-
TAP	128	-	-	-
FC (embedding)	128	-	-	-
Output	1211	-	-	-

Table 2: *Performance results in comparing the effect of L_2-normalization layer and length normalization strategy*

System Description	L_2-normalization layer	Length Normalization	Similarity Metric	C_{det}	$EER(\%)$
i-vector + Euclidean	N/A	✗	inner product	0.893	21.73
i-vector + cosine	N/A	✓	inner product	0.841	21.71
i-vector + PLDA	N/A	✗	PLDA	0.649	8.45
i-vector + l_2 norm + PLDA	N/A	✓	PLDA	0.643	8.20
Baseline end-to-end + Eucidean	✗	✗	inner product	0.758	7.42
Baseline end-to-end + cosine	✗	✓	inner product	0.553	5.48
Baseline end-to-end + PLDA	✗	✗	PLDA	0.524	5.90
Baseline end-to-end + l_2 norm + PLDA	✗	✓	PLDA	0.545	5.21
L_2-normalized end-to-end + Euclidean	✓	✗	inner product	**0.474**	5.10
L_2-normalized end-to-end + PLDA	✓	✗	PLDA	0.525	**4.93**

Table 3: *Verification performance on VoxCeleb for various α*

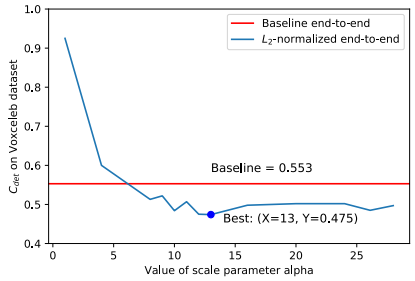
System Description	C_{det}	$EER(\%)$
End-to-end baseline	**0.553**	**5.48**
L_2-normalized (α=1)	0.925	10.11
L_2-normalized (α=4)	0.600	6.34
L_2-normalized (α=8)	0.513	5.48
L_2-normalized (α=11)	0.507	5.30
L_2-normalized (α=12)	0.475	5.16
L_2-normalized (α=13)	**0.474**	**5.10**
L_2-normalized (α=16)	0.498	5.31
L_2-normalized (α=20)	0.502	5.46
L_2-normalized (α=24)	0.502	5.52
L_2-normalized (α=28)	0.497	5.52
L_2-normalized (α trained=26.1)	0.485	5.50

In testing stage, all the testing utterances with different duration are tested on the same model. Because the duration is arbitrary, we feed the testing speech utterance to the trained neural network one by one.

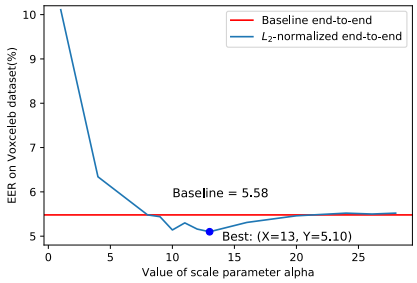
3.4. Evaluation

We first investigate scale parameter setting of α. In the feature comparison step, All the systems in Table 3 and Fig. 5 adopt the cosine similarity or equivalently L_2-normalized Euclidean distance to measure the similarities between features. The cosine similarity is similarity measure which is independent of magnitude. It can be seen as the normalized version of inner-product of two vectors. In Fig. 5, it shows that the end-to-end baseline system attains C_{det} of 0.553 and EER of 5.58%, whereas the proposed L_2-normalized end-to-end system achieves the best C_{det} of 0.475 and EER of 5.10%. According to Equation 5, for classes C of 1211 and probability score p of 0.9, the theoretical lower bound of α is 9. The performance is poor when α is below the lower bound and stable with α higher than the lower bound. The best α in our experiment is of 13, which is slightly larger than the lower bound.

We further explore effect of the L_2-normalization layer or length normalization strategy in the whole SV pipeline. The results are shown in Table 2. First, as expected, length normalization is a crucial step to boost the performance in both i-vector and baseline end-to-end system. No matter in i-vector or baseline end-to-end system, length normalization followed by PLDA scoring achieves the best performance. When it turns to L_2-normalized end-to-end system, since the feature embedding extracted from neural network has already been normlized



(a) C_{det} Performance



(b) EER Performance

Figure 5: *Performance tendency curve considering various α*

to unit length, additional length normalization is not required. In testing stage, a simple inner product achives the best perfomance of C_{det} 0.474 and EER 5.10%, even slightly better than PLDA scoring result of C_{det} 0.525 and EER 4.93%.

4. Conclusions

In this paper, a a simple L_2-normalization layer followed by a scale layer after the penultimate layer of the end-to-end neural network. Therefore, the neural network can learn the length-normalized feature embedding in an end-to-end manner. We conducted experiment on the verification task of Voxceleb dataset. The results show that integrating this simple yet elegant step in the end-to-end training pipeline significantly boosts the performance of speaker verification. In testing stage of L_2-normalized end-to-end system, a simple inner product can achieve state-of-the-art result.

5. References

[1] W. Campbell, D. Sturim, and D. Reynolds, "Support vector machines using gmm supervectors for speaker verification," *IEEE Signal Processing Letters*, vol. 13, no. 5, pp. 308–311, 2006.

[2] T. Kinnunen and H. Li, "An overview of text-independent speaker recognition: From features to supervectors," *Speech Communication*, vol. 52, no. 1, pp. 12–40, 2010.

[3] W. Cai, Z. Cai, W. Liu, X. Wang, and M. Li, "Insights into end-to-end learning scheme for language identification," in *ICASSP 2018*.

[4] W. Cai, Z. Cai, X. Zhang, X. Wang, and M. Li, "A novel learnable dictionary encoding layer for end-to-end language identification," in *ICASSP 2018*.

[5] D. A. Reynolds, "Speaker identification and verification using gaussian mixture speaker models," *Speech communication*, vol. 17, no. 1-2, pp. 91–108, 1995.

[6] N. Dehak, P. Kenny, R. Dehak, P. Dumouchel, and P. Ouellet, "Front-end factor analysis for speaker verification," *IEEE Transactions on Audio, Speech, and Language Processing*, vol. 19, no. 4, pp. 788–798, 2011.

[7] S. Prince and J. Elder, "Probabilistic linear discriminant analysis for inferences about identity," in *ICCV 2007*, pp. 1–8.

[8] P. Kenny, "Bayesian speaker verification with heavy tailed priors," *ODYSSEY 2010*.

[9] D. Garcia-Romero and C. Y. Espy-Wilson, "Analysis of i-vector length normalization in speaker recognition systems." in *Proc. INTERSPEECH*, 2011, pp. 249–252.

[10] P. Bousquet, A. Larcher, D. Matrouf, J. Bonastre, and O. Plchot, "Variance-spectra based normalization for i-vector standard and probabilistic linear discriminant analysis," in *Proc. Odyssey*, 2012.

[11] P. Matejka, O. Glembek, F. Castaldo, M. Alam, O. Plchot, P. Kenny, L. Burget, and J. Cernocky, "Full-covariance ubm and heavy-tailed plda in i-vector speaker verification," in *Proc. ICASSP*, 2011, pp. 4828–4831.

[12] E. Variani, X. Lei, E. McDermott, I. L. Moreno, and J. Gonzalez-Dominguez, "Deep neural networks for small footprint text-dependent speaker verification," in *Acoustics, Speech and Signal Processing (ICASSP), 2014 IEEE International Conference on*. IEEE, 2014, pp. 4052–4056.

[13] I. Lopez-Moreno, J. Gonzalez-Dominguez, O. Plchot, D. Martinez, J. Gonzalez-Rodriguez, and P. Moreno, "Automatic language identification using deep neural networks," in *ICASSP 2014*.

[14] D. Snyder, P. Ghahremani, D. Povey, D. Garcia-Romero, Y. Carmiel, and S. Khudanpur, "Deep neural network-based speaker embeddings for end-to-end speaker verification," in *SLT 2017*, pp. 165–170.

[15] J. Ma, Y. Song, I. Mcloughlin, W. Guo, and L. Dai, "End-to-end language identification using high-order utterance representation with bilinear pooling," in *INTERSPEECH 2017*, pp. 2571–2575.

[16] L. Chao, M. Xiaokong, J. Bing, L. Xiangang, Z. Xuewei, L. Xiao, C. Ying, K. Ajay, and Z. Zhenyao, "Deep speaker: an end-to-end neural speaker embedding system," 2017.

[17] C. Zhang and K. Koishida, "End-to-end text-independent speaker verification with triplet loss on short utterances," in *Proc. Interspeech 2017*, 2017, pp. 1487–1491. [Online]. Available: http://dx.doi.org/10.21437/Interspeech.2017-1608

[18] C. Zhang and K. K., "End-to-end text-independent speaker verification with flexibility in utterance duration," in *Automatic Speech Recognition and Understanding Workshop (ASRU), 2017 IEEE*, 2017, pp. 584–590.

[19] R. Ranjan, C. D. Castillo, and R. Chellappa, "L2-constrained softmax loss for discriminative face verification," *arXiv preprint arXiv:1703.09507*, 2017.

[20] F. Wang, X. Xiang, J. Cheng, and A. L. Yuille, "Normface: L2 hypersphere embedding for face verification," *arXiv preprint arXiv:1704.06369*, 2017.

[21] A. Nagrani, J. S. Chung, and A. Zisserman, "Voxceleb: a large-scale speaker identification dataset," in *INTERSPEECH*, 2017.

[22] C. Greenberg, "The NIST 2012 Speaker Recognition Evaluation Plan," *NIST Technical Report*, 2012.

[23] D. Povey and A. e. a. Ghoshal, "The kaldi speech recognition toolkit," in *ASRU 2011*.

[24] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *CVPR 2016*, 2016, pp. 770–778.